

Project Report: Java-Based Movie Ticket Booking System

Student Name: Mayank Gunwant

Registration Number: 25BAI10175

Institution: VIT Bhopal

Program: Artificial Intelligence and Machine Learning

1. Project Overview

This project involves the development of a fully functional, object-oriented Movie Ticket Booking System implemented in Java. The system is designed to simulate a real-world theater management and ticket reservation environment. It leverages core Java concepts to manage users, movies, theaters, seat availability, and the booking lifecycle through a unified command-line interface.

2. System Architecture & Module Breakdown

The application is structured around several distinct class entities, each handling a specific domain of the booking system.

2.1 User Management (`User.java` , `Admin.java` , `Customer.java`)

The system utilizes inheritance to manage different user roles.

- **User Base Class:** Encapsulates common attributes such as `userId` , `name` , and `email` . It includes basic validation to ensure the email contains an '@' symbol.
- **Admin Class:** Extends the `User` class and includes administrative capabilities, specifically the simulated `addMovie()` function.
- **Customer Class:** Extends the `User` class, representing end-users with the ability to interface with the booking system via the `bookTicket()` function.

2.2 Theater and Movie Logistics (`Theater.java` , `Movie.java` , `Show.java`)

These classes establish the physical constraints and offerings of the cinema.

- **Movie:** Stores detailed entity data including `movieId` , `title` , `duration` , and `language` . It validates that the duration is a positive integer.
- **Theater:** Represents a physical location, storing `theaterId` , `name` , `location` , and maintaining a dynamic list (`ArrayList`) of movies currently screening.
- **Show:** Acts as the binding class linking a `Movie` and a `Theater` . Crucially, it initializes an `ArrayList` of `Seat` objects (labeled A1 through A5) to track capacity for a specific screening.

2.3 Booking Engine (`Seat.java` , `Booking.java` , `BookingService.java`)

This module processes the transactional logic of reserving seats.

- **Seat:** A granular class representing individual seat states using a boolean `isBooked` flag.
- **BookingService:** The core transactional processor. It verifies seat existence within the `Show` , checks its `isBooked` status, locks the seat by calling `bookSeat()` , and generates a new `Booking` entity.
- **Booking:** A record-keeping class that links a specific `BookingId` with the respective `User` , `Show` , and `Seat` , capable of outputting a formatted confirmation receipt.

3. Technical Execution

The project successfully demonstrates core Object-Oriented Programming (OOP) concepts including Encapsulation, Inheritance, and Abstraction. Access modifiers ensure data integrity, such as preventing direct manipulation of the seat boolean states outside the designated booking methods. Polymorphism is highlighted through the overridden `performRole()` method in the derived User classes. The application runs through a persistent `Scanner`-driven `while` loop, providing a continuous, interactive console UI until the user explicitly exits.